

TRAINING WORKSHEET · MACHINE LEARNING METHODS

Boosted regression trees and XGBoost

How gradient boosting works, why XGBoost is the standard implementation, and how to fit and tune one sensibly in R.

EDAM Capacity Building Programme · Supplementary reading

1. The idea behind boosting

A single decision tree is easy to interpret but a weak predictor: shallow trees underfit, deep trees memorise noise. Boosting builds many deliberately weak trees in sequence, where each new tree is fitted to the errors left behind by all the trees before it. The final prediction is the sum of every tree's contribution, each scaled down by a **learning rate** so no single tree dominates.

This is **fundamentally different from a random forest**. A forest grows hundreds of independent trees in parallel on bootstrapped samples and averages them, reducing variance. **Boosting grows trees sequentially and dependently**, each correcting its predecessors, reducing bias as well. Boosting usually wins on predictive accuracy for tabular data, but it is more sensitive to over-fitting and needs more careful tuning.

“Gradient” boosting formalises this: rather than literally fitting to residuals, each tree is fitted to the **negative gradient of a chosen loss function**. That generalisation is what lets the same algorithm handle regression, classification, count data (Poisson) and survival outcomes by simply swapping the objective.

2. What XGBoost adds

XGBoost (eXtreme Gradient Boosting, Chen and Guestrin 2016) is a **specific implementation of gradient boosting** that became the default choice for **tabular problems**, for four main reasons:

- **Regularisation built into the objective** — it penalises both the number of leaves and the magnitude of the leaf weights, so over-fitting is controlled by the loss function itself rather than only by early stopping.
- **Second-order optimisation** — it uses both the gradient and the curvature (Hessian) of the loss, giving faster and more stable convergence than first-order gradient boosting.
- **Native handling of missing data** — at each split the algorithm learns a default direction for missing values, so no imputation step is required.
- **Speed** — parallelised split-finding, cache-aware data structures and approximate histogram-based splitting make it fast on large datasets.

3. The parameters that matter

Parameter	What it controls	Typical starting range
eta	Learning rate — how much each tree contributes. Lower means more trees but better generalisation.	0.01–0.1
nrounds	Number of boosting rounds (trees). Set high and let early stopping choose.	500–5000
max_depth	Depth of each tree — the main control on interaction complexity.	3–8
min_child_weight	Minimum summed instance weight in a leaf; higher values give more conservative trees.	1–10
subsample	Fraction of rows sampled per tree, adding stochasticity.	0.6–0.9
colsample_bytree	Fraction of features sampled per tree.	0.6–0.9
gamma	Minimum loss reduction required to make a further split.	0–5
lambda / alpha	L2 and L1 penalties on leaf weights.	lambda 1–10, alpha 0–1

Table 1. Core XGBoost hyperparameters. *eta* and *max_depth* do most of the work; tune those first.

The single most useful relationship to remember: *eta* and *nrounds* trade off directly. Halving the learning rate roughly doubles the number of rounds needed. Set a low *eta*, set *nrounds* generously, and let early stopping on a validation set decide when to stop. It is important to state your tuning parameters in your methods.

4. Fitting a model in R

The **xgboost** package requires a **numeric matrix**, so factors must be one-hot encoded first — `model.matrix()` or the `vtreat` package will do this. The `xgb.DMatrix` format is the efficient internal representation and is what the tuning functions expect.

```
dtrain <- xgb.DMatrix(data = X_train, label = y_train)
dvalid <- xgb.DMatrix(data = X_valid, label = y_valid)
params <- list(objective = "reg:squarederror", eta = 0.05, max_depth = 5,
               subsample = 0.8, colsample_bytree = 0.8, eval_metric = "rmse")
fit <- xgb.train(params, dtrain, nrounds = 3000,
                watchlist = list(valid = dvalid), early_stopping_rounds = 50)
```

Use `xgb.cv()` with the same parameter list to choose nrounds by cross-validation before fitting the final model on all training data. For a tidyverse workflow, the `tidymodels` framework wraps XGBoost through the `parsnip` and `tune` packages, which handles the resampling and grid search for you.

Common objectives: `reg:squarederror` for continuous outcomes, `binary:logistic` for two-class problems, `multi:softprob` for multi-class, and `count:poisson` for count data such as case numbers.

5. Interpreting the fitted model

- **Gain-based importance** — `xgb.importance()` ranks features by the improvement in loss their splits deliver. Quick, but it says nothing about the direction of an effect and is biased towards high-cardinality variables.
- **SHAP values** — the recommended approach. `predict(fit, X, predcontrib = TRUE)` returns exact SHAP values via `TreeExplainer`, and the `SHAPforxgboost` package turns these into summary, dependence and force plots. See the companion SHAP worksheet.
- **Partial dependence and ICE plots** — show the marginal effect of one feature across its range; the `pdp` and `DALEX` packages provide these.

“Importance tells you which features the model used. SHAP tells you how it used them.”

6. Practical cautions

- **Validate honestly** — early stopping uses the validation set, so that set is no longer untouched. Keep a separate test set or nest the tuning inside an outer cross-validation loop, for an unbiased performance estimate.
- **Respect structure in the data** — for time-series, split chronologically rather than randomly. For clustered data (patients within facilities, districts within regions), split by cluster or the model will appear far better than it is.
- **Class imbalance** — use `scale_pos_weight` rather than resampling where possible and evaluate with AUC-PR rather than accuracy when positives are rare.
- **Extrapolation** — tree ensembles cannot predict outside the range of the training outcomes. Predictions saturate at the edges of the observed data, which matters for projections under future scenarios.
- **Not causal** — as with any predictive model, high importance reflects predictive usefulness, not a causal effect.

References and further reading

- Chen, T. and Guestrin, C. (2016) ‘XGBoost: a scalable tree boosting system’, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.
- Friedman, J. H. (2001) ‘Greedy function approximation: a gradient boosting machine’, *Annals of Statistics*, 29(5), pp. 1189–1232.
- Elith, J., Leathwick, J. R. and Hastie, T. (2008) ‘A working guide to boosted regression trees’, *Journal of Animal Ecology*, 77(4), pp. 802–813.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The Elements of Statistical Learning*. 2nd edn. New York: Springer, ch. 10.
- Lundberg, S. M., Erion, G., Chen, H. et al. (2020) ‘From local explanations to global understanding with explainable AI for trees’, *Nature Machine Intelligence*, 2(1), pp. 56–67.
- Chen, T., He, T., Benesty, M. et al. (2024) *xgboost: Extreme Gradient Boosting*. R package. Available at: <https://cran.r-project.org/package=xgboost>
- XGBoost Developers (2024) XGBoost Parameters. Available at: <https://xgboost.readthedocs.io/en/stable/parameter.html>
- Kuhn, M. and Silge, J. (2022) *Tidy Modeling with R*. Sebastopol: O’Reilly. Available at: <https://www.tmw.org/>